

MODULE 07 · PART 2 - WINDOW FUNCTIONS

Aggregates as Windows

The same five functions you already know, in a position that changes what they mean.

PART 2 of 8 MODULE 7 of 26 RUNS about 32 min

An aggregate with an OVER clause
answers **how does this row compare**
rather than **what is the total**.

What changes when you add OVER

- ◆ **The row count stops changing.** One row in, one row out, so the grain you had is the grain you keep.
- ◆ **GROUP BY stops being required.** The partition does the grouping, and it is scoped to that one function call.
- ◆ **A frame appears if you add an ORDER BY,** and on most engines you did not choose it. Module five.
- ◆ **DISTINCT stops working,** on most engines, which is the second half of this module and the biggest surprise in it.

THE TAKEAWAY

Three of these are gains and one is a loss. The loss is the one nobody expects, because DISTINCT is the first thing people reach for when counting.

Three shapes, and what each is for

SHAPE	WHAT IT PRODUCES
<code>OVER (PARTITION BY k)</code>	The group total, on every row. A denominator.
<code>... ORDER BY t ROWS UNBOUNDED PRECEDING</code>	A running total, climbing to the group total.
<code>... ORDER BY t ROWS BETWEEN n PRECEDING AND CURRENT ROW</code>	A trailing window that forgets.
<code>OVER ()</code>	The grand total, on every row. The other denominator.

THE TAKEAWAY

Nearly every aggregate window is one of these four. The first and last are denominators, and the middle two are series.

No new functions in this module

- ◆ **SUM, AVG, COUNT, MIN and MAX.** The same five you have used since your first week, in a different position.
- ◆ **Which makes this the easiest module to read and the hardest to review,** because nothing on the page looks unfamiliar.
- ◆ **Every defect here is a denominator or a weighting,** and both produce a number rather than an error.
- ◆ **So the work is arithmetic rather than syntax.** Knowing what a figure divides by is the whole skill.

THE TAKEAWAY

A share, an average and a ratio all look like one line of obvious SQL. What they divide by is invisible, and that is where every scenario in this module lives.

Both forms, one query

```
1 SELECT store_key,  
2        SUM(net_amount) AS store_total,  
3        SUM(SUM(net_amount)) OVER () AS all_stores,  
4        COUNT(*) AS lines_in_store,  
5        SUM(COUNT(*)) OVER () AS lines_everywhere  
6 FROM fact_sales_line  
7 WHERE date_key = 20260831  
8 GROUP BY store_key;
```

5 SUM of COUNT, not COUNT of COUNT. The outer function has to add the group counts together.

THE TAKEAWAY

Line 5 is the correct version of module three's scenario one. `COUNT(*) OVER ()` there would return 412, and `SUM(COUNT(*)) OVER ()` returns the sales lines.

MIN and MAX as windows

- ◆ **MAX over a partition gives the group's best**, on every row, so 'is this row the best' is a comparison rather than a join.
- ◆ **MAX over a running frame gives a high-water mark**, which is how drawdown and peak-to-date measures are built.
- ◆ **Neither is FIRST_VALUE or LAST_VALUE**. Largest and earliest are different questions, as module six said.
- ◆ **Both skip nulls**, so a partition of entirely null values returns null rather than an error.

THE TAKEAWAY

MAX over a running frame is monotonic by definition, which is why module six's carry-forward substitution failed. Here that behaviour is the feature.

Percent of total. One expression, one pass,
and **no subquery**.

Share of region, share of nation

```
1 SELECT s.region_key, f.store_key, f.net_amount,  
2        f.net_amount  
3        / SUM(f.net_amount) OVER (PARTITION BY s.region_key)  
4        AS share_of_region,  
5        f.net_amount  
6        / SUM(f.net_amount) OVER () AS share_of_nation  
7 FROM fact_sales_line f  
8 JOIN dim_store s ON s.store_key = f.store_key  
9 WHERE f.date_key = 20260831;
```

- 3 Seven denominators, one per region. Every row divides by its own region's total.
- 9 Both denominators are totals of the rows that survived this WHERE. Neither is a total of the table.

THE TAKEAWAY

Two shares from one pass. The mark on line 9 is scenario one, and it is the most common defect in this entire family.

Every share has a denominator, and the WHERE decides it

- ◆ **The window runs at step five, after WHERE.** So the denominator is the filtered universe, always.
- ◆ **Filter to one region and 'share of nation' becomes share of region,** and every value sums to exactly 100%.
- ◆ **Which looks correct.** Shares summing to 100% is exactly what you would check, and it passes.
- ◆ **Compute the wide denominator before filtering,** in a CTE, or join it in as a scalar. Then the label is true.

THE TAKEAWAY

A share column is only as meaningful as its denominator, and the denominator is invisible in the output. Only the column name claims what it is.

A denominator that survives the filter

```
1 WITH national AS (  
2   SELECT SUM(net_amount) AS total_net  
3   FROM fact_sales_line  
4   WHERE date_key = 20260831  
5 )  
6 SELECT f.store_key, f.net_amount,  
7        f.net_amount / n.total_net AS share_of_nation  
8 FROM fact_sales_line f  
9 JOIN dim_store s ON s.store_key = f.store_key  
10 CROSS JOIN national n  
11 WHERE s.region_key = 3;
```

- 10 A cross join to a single-row CTE. One value, broadcast to every row, computed before the region filter.
- 11 The region filter now narrows the numerator and leaves the denominator alone. Shares no longer sum to 100%.

THE TAKEAWAY

The denominator is now stated in its own query block with its own filter, which means somebody reading it can see what population it covers.

Nested shares

```
1 SELECT s.region_key, f.store_key,  
2        SUM(f.net_amount) AS store_sales,  
3        SUM(SUM(f.net_amount)) OVER (PARTITION BY s.region_key)  
4        AS region_sales,  
5        SUM(SUM(f.net_amount)) OVER () AS national_sales,  
6        SUM(f.net_amount)  
7        / SUM(SUM(f.net_amount)) OVER (PARTITION BY s.region_key)  
8        AS share_of_region,  
9        SUM(SUM(f.net_amount)) OVER (PARTITION BY s.region_key)  
10       / SUM(SUM(f.net_amount)) OVER () AS region_share_of_nation  
11 FROM fact_sales_line f JOIN dim_store s USING (store_key)  
12 GROUP BY s.region_key, f.store_key;
```

10 Two shares at two levels. Multiply them and you get the store's share of the nation, exactly.

THE TAKEAWAY

Share of region times region share of nation equals share of nation. If those three columns do not multiply out, one of the denominators is not what its name says.

When shares do not sum to one

- ◆ **A join fanned out**, so some rows are counted twice in the numerator and twice in the denominator, unevenly.
- ◆ **The measure can be negative**. Returns make a share exceed one, and a negative group total flips every sign.
- ◆ **Nulls in the partition column** formed a partition of their own, which is module three's silent extra group.
- ◆ **Or they sum to one and should not**, which is scenario one and is the only one of these with no visible symptom.

THE TAKEAWAY

Three of these announce themselves by failing a sum-to-one check. The fourth passes that check, which is why it is the one that ships.

Window denominator, or joined

WINDOW

One pass

- No join, no extra scan
- Denominator is invisible
- Follows the WHERE clause
- Cannot be tested alone

JOINED CTE

Two blocks

- Costs a join, usually cheap
- Denominator is named and visible
- Has its own filter, stated
- Can be selected and checked

THE TIEBREAKER

The window is shorter and the CTE is auditable. For a denominator that appears in a published figure, auditable usually wins.

Percent of total on a nullable measure

- ◆ **SUM skips nulls, so the denominator excludes them.** The numerator on a null row is null, and the share is null.
- ◆ **Which means null rows have no share and do not reduce anybody else's.** The remaining shares still sum to one.
- ◆ **That is arithmetically consistent and often wrong.** A row with an unknown amount is not a row with no amount.
- ◆ **Decide before dividing.** Coalesce to zero, exclude the rows explicitly, or publish the null count alongside.

THE TAKEAWAY

The shares summing to one is the reassuring part and it is not evidence of anything. It would sum to one whether the nulls were two rows or two million.

A regional manager asks for a store report showing each store's share of national sales. The analyst filters the query to the manager's region for speed, and computes the share with `SUM(net_amount) OVER ()` as the denominator.

The report is delivered with a column headed 'share of national sales'. The manager checks it, notices the column sums to exactly 100.0%, and concludes the figures are sound. There are seven regions.

What is wrong?

- ☐ A The window needs `PARTITION BY region_key` in order to produce a national figure.
- ☐ B Nothing. Within a single-region extract, share of national and share of region are necessarily the same number.
- ☐ C The window sees only post-WHERE rows, so the denominator is the region total. It must be computed before filtering.
- ☐ D `SUM OVER ()` has no `ORDER BY`, so the denominator is non-deterministic and varies between runs.

◆ PAUSE AND DECIDE

What is wrong?

- A

 Add PARTITION BY region
Produces share of region explicitly, which is the number already being returned. It renames the defect rather than fixing it.
- B

 Nothing is wrong
They are the same number here and they are not the same measure. The column claims a national denominator that was never computed.
- C

 The filter set the denominator
Step two ran before step five, so the total covers one region. The column is mislabelled by a factor of roughly seven.
- D

 Non-deterministic denominator
A sum over a whole partition needs no ordering and is entirely deterministic. Module two does not apply to an unordered aggregate.

THE SIGNAL

"Sums to exactly 100.0%." Seven regions and a national share column that sums to one hundred is arithmetically impossible. The check that reassured the manager is the evidence.

Distance from the group

```
1 SELECT store_key, sale_date, daily_sales,  
2        AVG(daily_sales) OVER w          AS region_avg,  
3        daily_sales - AVG(daily_sales) OVER w AS vs_avg,  
4        daily_sales / NULLIF(AVG(daily_sales) OVER w, 0)  
5        * 100                             AS index_vs_avg  
6 FROM agg_store_day_region  
7 WINDOW w AS (PARTITION BY region_key, sale_date);
```

- 3 An absolute gap, in dollars. Comparable within a region and not across regions of different sizes.
- 5 A relative index, base 100. Comparable everywhere, and it hides the size of the underlying numbers.

THE TAKEAWAY

Both are correct and they answer different questions. The difference is dollars against average, versus percent of average, and only one travels between regions.

Index against average

- ◆ **Base 100 is the convention**, so 118 reads as eighteen percent above the comparison group and 92 as eight percent below.
- ◆ **It makes unlike things comparable**, which is why retail and market research use it everywhere.
- ◆ **And it removes the magnitude entirely**. A store at index 200 might be twice the average of a very small region.
- ◆ **Always publish the index with its base**. Index against what, over what period, is not recoverable from the number.

THE TAKEAWAY

An index is a ratio whose denominator has been hidden on purpose. That is the point of it, and it is also why the denominator has to be stated somewhere else.

An indexed series, base 100

```
1 SELECT store_key, sale_date, daily_sales,  
2        FIRST_VALUE(daily_sales) OVER (  
3            PARTITION BY store_key  
4            ORDER BY sale_date  
5            ROWS BETWEEN UNBOUNDED PRECEDING  
6                AND UNBOUNDED FOLLOWING) AS base_day,  
7        daily_sales * 100.0  
8        / NULLIF(FIRST_VALUE(daily_sales) OVER (  
9            PARTITION BY store_key ORDER BY sale_date  
10           ROWS BETWEEN UNBOUNDED PRECEDING  
11                AND UNBOUNDED FOLLOWING), 0) AS indexed  
12 FROM agg_store_day;
```

6 The whole-partition frame, because the base is the first row of the series rather than the first row up to here.

THE TAKEAWAY

Without the explicit frame this still works, because the default frame starts at UNBOUNDED PRECEDING. It works by accident, which module six warned about.

Share, or index

SHARE

Parts of a whole

- Sums to one across the group
- Denominator is the group total
- Adding a member changes everyone
- Answers how much of it is this

INDEX

Relative to typical

- Averages to about one hundred
- Denominator is the group average
- Adding a member barely moves it
- Answers how unusual is this

THE TIEBREAKER

Shares are zero-sum and indices are not. If a stakeholder is upset that their share fell while their sales rose, they wanted an index.

Opening and closing, on every row

```
1 SELECT store_key, sale_date, running_balance,  
2        FIRST_VALUE(running_balance) OVER w AS opening,  
3        LAST_VALUE(running_balance) OVER w AS closing,  
4        MIN(running_balance) OVER w AS lowest,  
5        MAX(running_balance) OVER w AS highest  
6 FROM vw_store_balance  
7 WINDOW w AS (  
8     PARTITION BY store_key, retail_month_key  
9     ORDER BY sale_date  
10    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```

8 The period is in the partition, so all four measures reset at every retail month boundary.

THE TAKEAWAY

Four period measures on one window definition. The retail month in the PARTITION BY is what makes them period measures rather than lifetime ones.

Five shapes worth knowing by name

- ◆ **Share of total.** Value over a partition sum. Sums to one within the group.
- ◆ **Index against average.** Value over a partition average, times a hundred. Averages to a hundred.
- ◆ **Running total.** An ordered sum with an unbounded preceding frame. Climbs to the group total.
- ◆ **Moving average.** An ordered average with a trailing frame. Reacts and forgets.
- ◆ **Conditional share.** A filtered numerator over an unfiltered denominator. Promotional share of trade.

THE TAKEAWAY

Five shapes cover almost every aggregate window written in analytics. Learning to recognise which one is being asked for is most of the job.

A window over a grouped result sees **groups**, not rows. That is the point, and it is also the trap.

Grouped, ranked and shared, in one pass

```
1 SELECT s.region_key, f.store_key,  
2        SUM(f.net_amount) AS store_sales,  
3        SUM(SUM(f.net_amount)) OVER (  
4          PARTITION BY s.region_key) AS region_sales,  
5        SUM(f.net_amount)  
6        / SUM(SUM(f.net_amount)) OVER (  
7          PARTITION BY s.region_key) AS share_of_region,  
8        RANK() OVER (PARTITION BY s.region_key  
9                     ORDER BY SUM(f.net_amount) DESC) AS rank_in_region  
10 FROM fact_sales_line f  
11 JOIN dim_store s ON s.store_key = f.store_key  
12 GROUP BY s.region_key, f.store_key;
```

4 The inner SUM runs at step three, per store. The outer SUM runs at step five, over the 412 grouped rows.

THE TAKEAWAY

412 rows out. Every window here operates on those 412, never on the 2.41 billion, which is why this is cheap as well as correct.

Reading a doubled aggregate, again

- ◆ Inner call, step three, one value per group. Outer call, step five, over the grouped rows.
- ◆ **COUNT(*) OVER ()** in a grouped query counts groups. Module three's scenario, and it is the same rule.
- ◆ **AVG(AVG(x)) OVER ()** is the average of averages, which is almost never the average, and looks entirely reasonable.
- ◆ **SUM and COUNT nest safely. AVG, MIN and MAX do not.** That asymmetry is worth memorising on its own.

THE TAKEAWAY

Sums of sums are sums. Averages of averages are not averages. The syntax gives no hint which of those you have written.

Two regional averages

AVERAGE OF AVERAGES

Every store weighted equally

```
1 AVG(AVG(basket_value)) OVER (  
2   PARTITION BY region_key)  
3  
4 -- a store with 40 baskets  
5 -- counts as much as one  
6 -- with 40,000
```

The unweighted mean of store means. A real measure, occasionally the one you want, and not the region average.

WEIGHTED

Every basket weighted equally

```
1 SUM(SUM(basket_value)) OVER (  
2   PARTITION BY region_key)  
3 / SUM(COUNT(*)) OVER (  
4   PARTITION BY region_key)  
5  
6 -- total value over total baskets
```

Total value divided by total baskets. This is the region's average basket, and it is what people mean.

THE TIEBREAKER

Rebuild the average from its parts. Sum the numerators, sum the denominators, then divide. That works at any level of nesting.

A regional performance pack shows average basket value per store, and alongside it the regional average for comparison. The query groups by store and computes the regional figure with `AVG(AVG(basket_value)) OVER (PARTITION BY region_key)`.

Finance notes that the regional averages in the pack are consistently about eleven percent higher than the same figure in the group finance report. Kestrel's regions each contain a handful of very large stores and a long tail of small convenience sites.

What is the cause?

- ☐ A The window needs `PARTITION BY store_key` as well, so each store's average is computed within itself.
- ☐ B Averaging averages weights every store equally. Divide summed value by summed basket count instead.
- ☐ C Basket values are skewed, so the mean is the wrong statistic and a median should be used at both levels.
- ☐ D The query should group by region directly rather than by store, which removes the nesting entirely.

◆ PAUSE AND DECIDE

What is the cause?

- A

 Partition by store as well
Puts each grouped row alone in its own partition, so the regional figure becomes that store's own average. Wrong, and no longer obviously so.
- B

 Unweighted mean of means
Small convenience sites have higher average baskets per store and far fewer baskets. Weighting them equally lifts the regional figure.
- C

 Use a median
A defensible argument about the statistic, and it does not explain a consistent gap against a report using the same statistic.
- D

 Group by region directly
Gives the correct regional average and loses the per-store rows the pack exists to show. Right number, wrong grain.

THE SIGNAL

"A handful of very large stores and a long tail of small ones." A consistent one-directional gap plus an unbalanced population is the signature of a weighting problem.

A share is **a claim about a population**. The SQL states the numerator and hides the denominator.

COUNT(DISTINCT x) OVER (...) is **not supported** on most engines. Including both of Kestrel's.

DISTINCT in a window, by engine

ENGINE	COUNT(DISTINCT X) OVER (...)
Amazon Redshift	No. The docs state DISTINCT is not supported.
PostgreSQL 18	No. 'DISTINCT is not implemented for window functions'.
SQL Server	No. A ranking trick is the standard workaround.
Snowflake, BigQuery	Partly. A partition only, with no ORDER BY and no frame.
Oracle	Yes, with a partition. Not over an ordered frame.

THE TAKEAWAY

Even where it works it works only for a whole partition. A running distinct count is not expressible as a window function anywhere in common use.

Seventeen years, and a patch in flight

- ◆ **The Postgres request dates from July 2009**, on the admin mailing list, from someone who hit the error mid-query.
- ◆ **A patch was in review in April 2026**, adding DISTINCT for window aggregates whose frame covers the whole partition.
- ◆ **Single-argument DISTINCT only**, and every other frame shape still rejected. Even the fix is deliberately narrow.
- ◆ **Which tells you why it is hard.** A distinct count over a sliding frame means re-deduplicating on every row.

THE TAKEAWAY

The narrowness of the proposed fix is the useful detail. Whole-partition DISTINCT is tractable. Running DISTINCT is a different problem, and that is why nobody has it.

The two-rank trick

```
1 SELECT store_key, sale_date, product_key,  
2         DENSE_RANK() OVER (PARTITION BY store_key, sale_date  
3                             ORDER BY product_key)  
4         + DENSE_RANK() OVER (PARTITION BY store_key, sale_date  
5                             ORDER BY product_key DESC)  
6         - 1 AS distinct_products  
7 FROM fact_sales_line;
```

6 Rank from the front plus rank from the back, minus one, equals the number of distinct values in the partition.

THE TAKEAWAY

It works, it is portable, and it counts NULL as a value. COUNT(DISTINCT) does not, so on a nullable column this over-counts by exactly one.

Where that matters at Kestrel

- ◆ **promo_key is null on 62% of sales lines.** Every store-day has at least one null, so the trick over-counts every single partition.
- ◆ **Always by exactly one**, which makes it plausible and stable, and therefore very hard to notice.
- ◆ **Subtract a case for the null**, or exclude nulls before ranking. Both are one line and both need saying out loud.
- ◆ **On a NOT NULL column the trick is exact.** product_key is not null, so the previous slide is correct as written.

THE TAKEAWAY

An off-by-one that is constant across every partition will reconcile against nothing and look like a definitional difference. Those are the hardest to find.

The portable answer: two steps

```
1 WITH per_day AS (  
2   SELECT store_key, sale_date,  
3          COUNT(DISTINCT product_key) AS distinct_products  
4   FROM fact_sales_line  
5   GROUP BY store_key, sale_date  
6 )  
7 SELECT f.*, d.distinct_products  
8   FROM fact_sales_line f  
9   JOIN per_day d  
10      ON d.store_key = f.store_key  
11      AND d.sale_date = f.sale_date;
```

- 3 COUNT(DISTINCT) as an ordinary aggregate, which every engine supports and which handles nulls correctly.
- 9 Joined back on the grouping key, which is unique in per_day, so the join cannot fan out.

THE TAKEAWAY

Two query blocks instead of one expression. It is exact, it is portable, it handles nulls, and the intermediate result can be tested on its own.

Approximate distinct counts

- ◆ **APPROX_COUNT_DISTINCT and HLL exist on most warehouses**, and Snowflake documents theirs as available as a window function.
- ◆ **With a partition only.** No ORDER BY and no explicit frame, which is the same restriction as the exact version.
- ◆ **Accuracy is usually within a couple of percent**, which is fine for a dashboard tile and not for anything reconciled.
- ◆ **Never for a figure somebody will check by hand.** An approximate answer that disagrees with a count is a support ticket.

THE TAKEAWAY

Approximate counting solves the cost problem and not the expressiveness one. It still cannot give you a running distinct count over a frame.

An analyst needs, on every sales line, the number of distinct promotions running in that store on that day. They write `COUNT(DISTINCT promo_key) OVER (PARTITION BY store_key, sale_date)`. Redshift rejects the query.

They find the two-rank technique online, apply it, and the figures look plausible. A colleague checks a single store-day by hand against the promotions calendar and finds the report says four where the calendar says three.

What should they do?

- A

 Move `DISTINCT` outside the parentheses. Redshift accepts the keyword in that position on window aggregates.
- B

 Run the query on the Postgres replica instead, which supports `DISTINCT` inside window functions.
- C

 Keep the two-rank technique. One-off hand checks are unreliable and the promotions calendar is not the system of record.
- D

 Compute the distinct count as a grouped aggregate and join it back, which is exact and handles the nulls.

◆ PAUSE AND DECIDE

What should they do?

- A

 Move DISTINCT outside
There is no position in which Redshift accepts it. The documentation states plainly that DISTINCT is not supported on window aggregates.
- B

 Use the Postgres replica
Postgres 18 rejects it too, with its own error message. A patch existed in April 2026 and this replica runs 16.
- C

 Trust the trick
The hand check is right. DENSE_RANK counts null as a value, and 62% of lines have no promotion, so every partition reads one too high.
- D

 Grouped aggregate, joined back
COUNT(DISTINCT) as an ordinary aggregate excludes nulls correctly, and the join on a unique key cannot fan out.

THE SIGNAL

"Four where the calendar says three." Off by exactly one, on a column that is null on most rows. That pairing names the cause before anybody opens the query.

A moving average with its denominator

```
1 SELECT store_key, sale_date, daily_sales,  
2        AVG(daily_sales) OVER w AS avg_28d,  
3        COUNT(daily_sales) OVER w AS days_counted,  
4        COUNT(*) OVER w AS rows_in_frame  
5 FROM agg_store_day  
6 WINDOW w AS (  
7     PARTITION BY store_key ORDER BY sale_date  
8     ROWS BETWEEN 27 PRECEDING AND CURRENT ROW);
```

- 3 Days with an actual value. This is AVG's real denominator.
- 4 Rows in the frame. If these two disagree, some days were null and the average silently used fewer.

THE TAKEAWAY

Two counts rather than one. Their difference is the number of days the average quietly dropped, and it is invisible in every report that omits it.

A weighted moving average

```
1 SELECT store_key, sale_date,  
2        SUM(daily_sales * w.weight) OVER win  
3        / NULLIF(SUM(w.weight) OVER win, 0) AS weighted_avg  
4 FROM agg_store_day a  
5 JOIN dim_dow_weight w ON w.day_of_week = a.day_of_week  
6 WINDOW win AS (  
7   PARTITION BY store_key ORDER BY sale_date  
8   ROWS BETWEEN 6 PRECEDING AND CURRENT ROW);
```

- 3 Sum of value times weight, over sum of weight. The denominator must be the weights in the same frame, not the row count.

THE TAKEAWAY

AVG cannot express a weighted average. It always divides by a count, so a weighted one is always written as two SUMs and a division.

Cumulative, moving, or to-date

- ◆ **Cumulative average.** UNBOUNDED PRECEDING to CURRENT ROW. It converges and stops reacting to recent change.
- ◆ **Moving average.** n PRECEDING to CURRENT ROW. It reacts, and it forgets what happened before the window.
- ◆ **Period to date.** PARTITION BY the period, then UNBOUNDED PRECEDING. It resets at every period boundary.
- ◆ **All three are 'the average so far' in English.** They are three different series and people ask for them with one phrase.

THE TAKEAWAY

Month-to-date is a partition, not a frame. Putting the period in the PARTITION BY is what makes the series reset, and forgetting that gives a cumulative average instead.

Aggregating only some of the rows in a window, **without filtering any of them out.**

FILTER, or CASE

FILTER

Says what it means

```
1 SUM(net_amount)
2   FILTER (WHERE promo_key IS NOT NULL)
3   OVER (PARTITION BY store_key)
4   AS promo_sales
```

Postgres, SQLite and DuckDB. Reads as a sentence, and is rejected everywhere else.

CASE

Runs anywhere

```
1 SUM(CASE WHEN promo_key IS NOT NULL
2       THEN net_amount END)
3   OVER (PARTITION BY store_key)
4   AS promo_sales
```

Every engine. The missing ELSE makes non-matching rows null, and SUM skips nulls.

THE TIEBREAKER

The CASE version works because of module two. No ELSE means null, SUM skips nulls, and the non-matching rows contribute nothing while still existing.

Four measures, one window, one pass

```
1 SELECT store_key, sale_date, net_amount,  
2        SUM(net_amount) OVER w AS day_total,  
3        SUM(CASE WHEN promo_key IS NOT NULL  
4              THEN net_amount END) OVER w AS promo_total,  
5        COUNT(CASE WHEN promo_key IS NOT NULL  
6                THEN 1 END) OVER w AS promo_lines,  
7        SUM(CASE WHEN promo_key IS NOT NULL  
8              THEN net_amount END) OVER w  
9        / NULLIF(SUM(net_amount) OVER w, 0) AS promo_share  
10 FROM fact_sales_line  
11 WINDOW w AS (PARTITION BY store_key, sale_date);
```

- 11 One window definition, four measures. The engine partitions once and computes all four in the same pass.

THE TAKEAWAY

Promotional share of the day's trade, on every line, with no subquery and no self-join. This is the shape most useful window queries take.

Conditional aggregation, three warnings

- ◆ **COUNT(CASE ... THEN 1 END)** counts matches. **COUNT(*)** counts rows. The second ignores your condition entirely.
- ◆ **A missing ELSE 0 changes an AVG.** SUM skips the nulls, and AVG skips them from its denominator too.
- ◆ **So for AVG, write ELSE 0 if non-matching rows should count as zero**, and omit it if they should not count at all.
- ◆ **That choice is invisible in the output**, and it is the same denominator problem as module two's discount average.

THE TAKEAWAY

With SUM, the ELSE makes no difference. With AVG and COUNT it changes the answer. Most people learn the pattern on SUM and carry the habit across.

What aggregate windows cost

- ◆ **Windows sharing a specification share a sort.** Four measures on one WINDOW clause cost close to one measure.
- ◆ **Windows with different specifications do not.** Nine different OVER clauses can mean nine sorts and nine redistributions.
- ◆ **An ordering you did not need creates a distinct specification,** so an idle ORDER BY can cost a whole extra sort.
- ◆ **Aggregate first where the grain allows.** A window over 412 grouped rows and one over 2.41 billion are different queries.

THE TAKEAWAY

The named WINDOW clause is not only a readability device. Writing one definition makes it structurally impossible to accidentally create two specifications.

Testing a denominator

- ◆ **Assert the shares sum to one**, which catches fan-out and null partitions and does not catch a filtered denominator.
- ◆ **Assert the nested shares multiply out**, which does catch a filtered denominator, because the levels disagree.
- ◆ **Assert the denominator against a known total** from a different query or a different system. One row, one number.
- ◆ **Three assertions, three failure modes.** The first is the one everybody writes and the weakest of the three.

THE TAKEAWAY

Shares summing to one is necessary and not sufficient. It is satisfied by a denominator covering any population at all, including the wrong one.

The denominator assertion

```
1 SELECT SUM(share_of_nation)           AS should_be_1,  
2     COUNT(*) FILTER (  
3       WHERE share_of_nation IS NULL) AS null_shares,  
4     MAX(ABS(share_of_region  
5       * region_share_of_nation  
6       - share_of_nation))           AS should_be_0  
7 FROM report;
```

6 The strong one. If the two levels do not multiply out, one denominator covers a different population from the other.

THE TAKEAWAY

Line 1 would have passed in scenario one. Line 6 would not, because the regional denominator and the national denominator would have been the same number.

An hourly store dashboard computes nine measures over the sales fact, each with its own OVER clause. Six of them partition by store and day. Three of those six also carry an ORDER BY that no frame or ranking uses, copied from an earlier query.

The dashboard takes four minutes and the requirement is under one. Nightly materialisation has been ruled out because the dashboard is used during trading to react to the current day.

What is the first change to make?

- A Align the specifications that are semantically the same, name the window once, and drop the orderings nothing uses.
- B Remove the measures the business looks at least often, reducing the number of windows to compute.
- C Increase the cluster size, since nine sorts over a large fact table are inherently memory-bound.
- D Materialise the dashboard nightly, which removes the query from the critical path entirely.

◆ PAUSE AND DECIDE

What is the first change to make?

- A

 Align and name the windows
Six measures become one specification and one sort. The three idle orderings were each creating a distinct specification for nothing.
- B

 Remove measures
Trades function for speed before any engineering has been attempted, and the windows that remain still carry the same duplicated specifications.
- C

 Bigger cluster
Makes nine sorts faster rather than making them one sort. It also pays for the three that exist only because of a copied clause.
- D

 Materialise nightly
Explicitly ruled out by the requirement. The dashboard exists to show the current trading day, which a nightly build cannot do.

THE SIGNAL

"An ORDER BY that no frame or ranking uses." A clause doing nothing semantically is still doing something to the plan, and it is the cheapest thing on this slide to remove.

Writing an aggregate window, four questions

01

Shape

Denominator, running series, or trailing window? That picks the partition and the frame.

02

Population

What does the denominator cover, and does the WHERE clause agree with the column name?

03

Weighting

If it is an average, are you averaging values or averaging averages? Rebuild from parts if unsure.

04

Distinct

If the word distinct appears, stop. It is a grouped query joined back, on both of Kestrel's engines.

THE TAKEAWAY

Step two is the one that produces published numbers that are wrong. The other three produce errors or figures somebody eventually reconciles.

The four denominators

DENOMINATOR	WHAT IT MAKES THE FIGURE MEAN
SUM(x) OVER ()	Share of everything the query could see.
SUM(x) OVER (PARTITION BY k)	Share of this row's group.
A joined CTE total	Share of a population you defined and can inspect.
COUNT(x) OVER (...)	An average's hidden denominator, excluding nulls.

THE TAKEAWAY

Four denominators and only one of them is written where a reader can check it. That is the argument for the joined form whenever the figure is published.

The confusions, settled

SYMPTOM	THE CAUSE
Shares sum to 100% when they should not	The denominator was filtered by the same WHERE.
A regional average consistently above the finance one	Average of averages, with unequal group sizes.
COUNT OVER () returns the number of groups	The query has a GROUP BY. The window ran at step five.
A distinct count one higher than the truth	The two-rank trick on a nullable column.
DISTINCT rejected in a window	Correct behaviour. Almost no engine implements it.
Nine windows, nine sorts	Nine specifications. Some of them differ only by an idle ORDER BY.

THE TAKEAWAY

The first two are published wrong numbers. The rest announce themselves. That ratio is why the population question comes before all the others.

Five habits

- ◆ **Name the denominator's population in the column name**, and then check the WHERE clause agrees with it.
- ◆ **Never nest AVG inside AVG**. Rebuild it as summed numerator over summed denominator, every time.
- ◆ **Use the named WINDOW clause once you have two measures** sharing a partition. It saves a sort and prevents drift.
- ◆ **Reach for the grouped-and-joined form for any distinct count**, rather than the trick, unless the column is NOT NULL.
- ◆ **Delete any ORDER BY inside an OVER that nothing uses**. It is free to remove and it costs a sort to keep.

THE TAKEAWAY

The first habit is the only one that prevents a wrong published number. The other four prevent slow queries and quiet drift, which are cheaper problems.

You can compute anything about a row now. **Module eight** is about filtering on it.